

Performance Testing

Date	4 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust, Browser-based Developer Tools
Type of Testing	Load Testing, Stress Testing
Target Module / API	/recommend, /suitability, /research, /api/stats
Test Environment	Local Host Server (Python 3.12, Flask)
Test Date	4 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load test on crop recommendation endpoint (/recommend) under concurrent user traffic.	50	60	Response time <= 1 second, zero errors.
2	Stress test on suitability assessor engine (/suitability) with invalid input data.	25	30	Inputs validated, error messages rendered under 1.5 seconds.
3	Load test on research dashboard page (/research) loading multiple Chart.js libraries.	10	30	Static resources load and plots render successfully.
4	API performance test on stats retrieval endpoint (/api/stats).	50	60	Cached JSON returned under 200 milliseconds.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	9.33 milliseconds	Pass	Extremely low latency on local loopback interface.
2	Response Time (Max)	< 5 seconds	52 milliseconds	Pass	Peak spike during concurrent POST requests.
3	Throughput (Req/sec)	> 10 req/sec	24.6 req/sec	Pass	Stable throughput on Flask development web server.
4	Error Rate	< 1%	0%	Pass	All swarmed requests completed successfully (1,297 total).
5	CPU Utilization	< 80%	12%	Pass	Lightweight Python process execution.
6	Memory Utilization	< 80%	48 MB	Pass	Low RAM footprint with static JSON metrics aggregation.

Step 4: Observations & Analysis

Key Findings:

- Average response times under 50 concurrent virtual users remain at 9.33 milliseconds, which is well below the target limit of 2 seconds.
- Throughput peaked at 24.6 requests per second on the local development environment without connection drops.
- Error rate remained at 0% across all concurrent test runs.

Bottlenecks Identified:

- No significant processing bottlenecks were observed for standard user loads.
- For extremely high concurrent traffic (>10,000 requests), local caching of static assets can be optimized using CDN-based assets to reduce server latency.

Optimization Steps Taken:

- Pre-aggregated crop statistics into crop_stats.json to enable instant retrieval and rendering of visual metrics without querying raw CSV dataset entries.
- Added strict parameter range sanitization on the server backend to prevent CPU overhead from bad inputs.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

